

Finite Element Solver for the Two-Dimensional Wave Equation

Numerical Methods for Partial Differential Equations

Maksim Sukhodolskii

Contents

1	Introduction	2
2	Continuous Problem	2
2.1	Weak Formulation	2
2.2	Manufactured Standing Wave	3
3	Spatial Discretization	3
4	Time Discretization	4
4.1	The θ -Method	4
4.2	Newmark Methods	5
5	Implementation	6
5.1	Code Structure	6
5.2	Diagnostics and Output	7
6	Convergence Study	7
7	Dissipation and Dispersion	10
8	Additional Simulations	12
9	Scalability	12
10	Limitations and Possible Improvements	13
11	Conclusions	14

1 Introduction

The wave equation is a second-order hyperbolic partial differential equation. In two space dimensions it models, for example, the transverse displacement of a membrane, acoustic pressure waves, or idealized elastic vibrations. Its numerical approximation is less forgiving than the approximation of parabolic problems: besides consistency and stability, one must also control non-physical dissipation and dispersion. Numerical dissipation changes the amplitude of the wave, while numerical dispersion changes its phase velocity and gradually shifts the wave out of phase with the exact solution.

The project goal was to build a finite element solver for

$$\begin{cases} \frac{\partial^2 u}{\partial t^2} - c^2 \Delta u = f, & \text{in } \Omega \times (0, T], \\ u = g, & \text{on } \partial\Omega \times [0, T], \\ u(\cdot, 0) = u_0, \quad \frac{\partial u}{\partial t}(\cdot, 0) = u_1, & \text{in } \Omega, \end{cases} \quad (1)$$

where $\Omega \subset \mathbb{R}^2$ is a bounded domain and $c > 0$ is the wave speed. The implemented code supports several domains and boundary data through configuration files: standing waves on the square $[0, 5]^2$, Gaussian pulse propagation, driven boundaries, a circle, a rectangular domain and an donut. The numerical experiments focus on the square domain because it admits a simple exact solution and therefore allows a clean convergence and energy analysis.

The solver is written in C++ with deal.II and distributed linear algebra through Trilinos/MPI. Meshes are generated by Gmsh and stored in the `mesh/` directory. Experiment definitions are stored in `configs/`. The implementation writes ParaView-compatible `.vtu/.pvtu` files and CSV diagnostics used to produce the plots in `results/`.

2 Continuous Problem

2.1 Weak Formulation

For the weak formulation we use the Sobolev space

$$H^1(\Omega) = \left\{ v \in L^2(\Omega) : \frac{\partial v}{\partial x_i} \in L^2(\Omega), \ i = 1, 2 \right\},$$

where the derivatives are understood in the distributional (weak) sense. The space $H_0^1(\Omega)$ is the subspace of $H^1(\Omega)$ with zero trace on the boundary:

$$H_0^1(\Omega) = \{ v \in H^1(\Omega) : v = 0 \text{ on } \partial\Omega \}.$$

For homogeneous Dirichlet data, the natural test space is therefore

$$V = H_0^1(\Omega).$$

Multiplying (1) by $v \in V$, integrating over Ω , and using Green's formula gives

$$\int_{\Omega} u_{tt} v \, dx + c^2 \int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx \quad \forall v \in V. \quad (2)$$

The boundary integral vanishes because $v = 0$ on $\partial\Omega$. If non-homogeneous Dirichlet data are prescribed, one may introduce a lifting $R_g(\cdot, t)$ such that $R_g = g$ on the boundary and write $u = \hat{u} + R_g$ with $\hat{u} \in H_0^1(\Omega)$. The same weak equation then holds for the homogeneous unknown $\hat{u}$, with additional known terms on the right-hand side. In the code this idea is enforced algebraically: boundary values are interpolated at the boundary degrees of freedom and imposed strongly in the linear systems. This is the standard Cauchy-Dirichlet weak formulation for the wave equation. Salsa writes the same weak form using $\ddot{u}$ as a notation for u_{tt} [2].

The energy of the unforced homogeneous problem is

$$E(t) = \frac{1}{2} \int_{\Omega} |u_t|^2 \, dx + \frac{c^2}{2} \int_{\Omega} |\nabla u|^2 \, dx. \quad (3)$$

For $f = 0$ and fixed homogeneous boundary values, differentiating (3) and integrating by parts shows $E'(t) = 0$, this energy conservation property is one of the main qualitative features that a good wave solver should preserve or approximate well. This energy calculation is also the basis of the classical a priori estimates for the Cauchy-Dirichlet wave problem [2].

The balance is also useful for interpreting the driven-boundary examples. If the boundary is not fixed, the same calculation gives

$$\frac{dE}{dt} = \int_{\Omega} f u_t \, dx + c^2 \int_{\partial\Omega} \partial_n u u_t \, ds. \quad (4)$$

The volume term is the work of the source and the boundary term is the energy flux through the boundary. Therefore an increase of the energy in the driven left-boundary or two-sided-boundary videos is expected if the prescribed boundary displacement performs work on the membrane. Conversely, in the homogeneous standing-wave diagnostics, the correct reference behavior is constant energy, deviations from this reference isolate algorithmic dissipation or instability.

2.2 Manufactured Standing Wave

The main verification test uses the exact solution

$$u_{\text{ex}}(x, y, t) = \sin\left(\frac{\pi x}{5}\right) \sin\left(\frac{\pi y}{5}\right) \cos\left(\frac{c\pi\sqrt{2}}{5}t\right) \quad (5)$$

on $\Omega = [0, 5]^2$. This function vanishes on the boundary, so the boundary condition is homogeneous. The corresponding initial data and forcing are implemented in `function_factory.cpp`. The forcing is computed as $f = (u_{\text{ex}})_{tt} - c^2 \Delta u_{\text{ex}}$, which makes the analytical solution exactly compatible with (1).

3 Spatial Discretization

Let $\mathcal{T}_h$ be a triangular mesh of Ω and let $V_h \subset H^1(\Omega)$ be the continuous finite element space generated by deal.II's `FE_SimplexP`. In the performed experiments the finite element degree is $r = 1$, hence the expected best asymptotic behavior is $O(h^2)$ in the L^2 norm and $O(h)$ in the H^1 norm when the temporal error is sufficiently small.

Using the basis $\{\varphi_i\}_{i=1}^{N_h}$ and writing

$$u_h(x, t) = \sum_{j=1}^{N_h} U_j(t) \varphi_j(x),$$

the semi-discrete problem takes the matrix form

$$\mathbf{M}\ddot{\mathbf{u}}(t) + c^2 \mathbf{K}\mathbf{u}(t) = \mathbf{f}(t), \quad (6)$$

with

$$M_{ij} = \int_{\Omega} \varphi_j \varphi_i \, dx, \quad K_{ij} = \int_{\Omega} \nabla \varphi_j \cdot \nabla \varphi_i \, dx, \quad F_i(t) = \int_{\Omega} f(t) \varphi_i \, dx. \quad (7)$$

In the implementation the stiffness matrix stores only the geometric bilinear form $\int \nabla \varphi_j \cdot \nabla \varphi_i$, the factor c^2 is inserted in the time-stepping systems and in the energy calculation. This keeps the assembled matrices reusable when the wave speed changes.

Table 1: Meshes used by the project. The first four square meshes are used in the convergence runs. Finer meshes are used for temporal convergence and scalability.

Mesh	Domain	Nodes	Elements
square_5x5_h0p5.msh	$[0, 5]^2$	142	282
square_5x5_h0p25.msh	$[0, 5]^2$	511	1020
square_5x5_h0p125.msh	$[0, 5]^2$	1934	3866
square_5x5_h0p0625.msh	$[0, 5]^2$	7554	15106
square_5x5_h0p03125.msh	$[0, 5]^2$	29978	59954
square_5x5_h0p008.msh	$[0, 5]^2$	452660	905322
rectangle_8x5_h0p125.msh	$[0, 8] \times [0, 5]$	3079	6156
circle_h0p125.msh	disk centered at (2.5, 2.5)	1594	3186
donut_h0p125.msh	donut	1482	2964

The matrices are assembled cell by cell using Gaussian quadrature of order `fe_degree + 1`. The triangulation is first read from a Gmsh `.msh` file, then partitioned across MPI ranks and converted into a deal.II fully distributed triangulation. Locally owned and locally relevant degree-of-freedom sets are stored separately, which is essential for point evaluation, output and interface-cell computations in a distributed mesh.

4 Time Discretization

Two time discretization families are implemented. The θ -method is applied after rewriting the second-order equation as a first-order system, whereas the Newmark method acts directly on (6).

4.1 The θ -Method

Introduce $\mathbf{v} = \dot{\mathbf{u}}$. The first-order system is

$$\dot{\mathbf{u}} = \mathbf{v}, \quad \mathbf{M}\dot{\mathbf{v}} + c^2\mathbf{K}\mathbf{u} = \mathbf{f}. \quad (8)$$

The θ -method updates $\mathbf{u}^n, \mathbf{v}^n$ to $\mathbf{u}^{n+1}, \mathbf{v}^{n+1}$ by

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t \left((1 - \theta)\mathbf{v}^n + \theta\mathbf{v}^{n+1} \right), \quad (9)$$

$$\mathbf{M} \frac{\mathbf{v}^{n+1} - \mathbf{v}^n}{\Delta t} + c^2\mathbf{K} \left((1 - \theta)\mathbf{u}^n + \theta\mathbf{u}^{n+1} \right) = (1 - \theta)\mathbf{f}^n + \theta\mathbf{f}^{n+1}. \quad (10)$$

Substitution of (9) into (10) produces the system actually used in the code:

$$\left(\mathbf{M} + c^2\theta^2\Delta t^2\mathbf{K} \right) \mathbf{v}^{n+1} = R_\theta^n, \quad (11)$$

where R_θ^n contains only known quantities. Once the velocity is known, (9) gives the displacement. The implemented choices are:

$$\theta = 0 \quad \text{Forward Euler}, \quad \theta = \frac{1}{2} \quad \text{Crank-Nicolson}, \quad \theta = 1 \quad \text{Backward Euler}.$$

Forward Euler is explicit in the stiffness part, but it is not stable for the oscillatory wave dynamics considered in this report. Crank-Nicolson is second-order accurate and energy-conserving for the linear homogeneous problem. Backward Euler is first-order accurate but strongly dissipative. It is robust but not well suited to long-time wave propagation if the physical amplitude is important.

4.2 Newmark Methods

The Newmark family advances displacement, velocity and acceleration:

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \Delta t \mathbf{v}^n + \Delta t^2 \left(\left(\frac{1}{2} - \beta \right) \mathbf{a}^n + \beta \mathbf{a}^{n+1} \right), \quad (12)$$

$$\mathbf{v}^{n+1} = \mathbf{v}^n + \Delta t \left((1 - \gamma) \mathbf{a}^n + \gamma \mathbf{a}^{n+1} \right). \quad (13)$$

The new acceleration satisfies

$$\mathbf{M} \mathbf{a}^{n+1} + c^2 \mathbf{K} \mathbf{u}^{n+1} = \mathbf{f}^{n+1}.$$

For $\beta > 0$, substituting (12) gives a linear system for the new displacement:

$$\left(\frac{1}{\beta \Delta t^2} \mathbf{M} + c^2 \mathbf{K} \right) \mathbf{u}^{n+1} = \mathbf{R}_N^n. \quad (14)$$

The acceleration and velocity are then recovered algebraically. The two implemented variants are:

$$\beta = 0, \gamma = \frac{1}{2} \quad \text{leapfrog/central difference}, \quad \beta = \frac{1}{4}, \gamma = \frac{1}{2} \quad \text{avg accel.}$$

The leapfrog case is conditionally stable and very sensitive to the restriction $\Delta t \leq Ch$. The avg accel method is unconditionally stable for the linear model and, as confirmed by the diagnostics, conserves the discrete energy up to solver tolerance.

For the linear undamped second-order system considered here, Crank-Nicolson applied to the first-order system $(\mathbf{u}, \mathbf{v})$ is algebraically equivalent to the Newmark avg accel method with $\beta = 1/4$ and $\gamma = 1/2$. Both are the trapezoidal rule written in different variables. The code implements them through different linear systems, so small differences could appear from boundary handling or solver tolerances, but the mathematical time discretization is the same for the homogeneous standing-wave tests.

Table 2: Time integrators used in the experiments.

Code name	Parameters	Type	Expected time order	Qualitative behavior
Forward Euler	$\theta = 0$	explicit	$O(\Delta t)$	not stable for wave propagation; fails in tests
Backward Euler	$\theta = 1$	implicit	$O(\Delta t)$	unconditionally stable, highly dissipative
Crank-Nicolson	$\theta = 1/2$	implicit	$O(\Delta t^2)$	unconditionally stable; little/no dissipation; dispersive
Newmark leapfrog	$\beta = 0, \gamma = 1/2$	explicit with mass lumping	$O(\Delta t^2)$	conditionally stable, $\Delta t \leq Ch$
Newmark avg accel	$\beta = 1/4, \gamma = 1/2$	implicit	$O(\Delta t^2)$	CN-type; unconditionally stable; dispersive

The stability limitation of explicit schemes is not only a formal statement. The numerical domain of dependence must be wide enough to contain the physical domain of dependence. If the step is too large compared with the mesh size and the wave speed, high-frequency components are amplified and the energy grows rapidly. This is why the same spatial discretization may look accurate for a small enough time step and completely fail for a larger one. Implicit schemes avoid this blow-up, but they still differ qualitatively: Backward Euler suppresses high frequencies by damping, while Crank-Nicolson and Newmark avg accel keep the energy but can shift the phase of the wave when Δt is large. This distinction between amplification error and dispersion error is exactly the type of analysis used by Quarteroni for finite element schemes for hyperbolic equations [1].

5 Implementation

5.1 Code Structure

The executable is configured entirely through text files. Each run specifies the mode, mesh, finite element degree, wave speed, time step, number of steps, initial data, forcing, boundary data and time integration parameters. Relative paths are resolved with respect to the directory of the configuration file, which makes the examples reproducible from any working directory.

The main classes are:

- `WaveProblemConfig`, which parses and validates the configuration;
- `WaveSolver`, which owns the mesh, DoFHandler, matrices, vectors, boundary enforcement, output and diagnostics;
- `ThetaTimeIntegrator` and `NewmarkTimeIntegrator`, which implement the two time stepping families;
- `ConvergenceRunner`, which repeats solves over mesh or time-step lists and writes CSV files with errors and observed orders;
- `function_factory.cpp`, which maps names such as `standing_wave_5x5`, `gaussian_pulse` or `two_sided_driven_boundary` to deal.II functions.

Table 3: Representative configurations present in the repository.

Configuration family	Domain/mesh	c	Δt	Main purpose
<code>standing_wave</code>	square, $h = 0.0625$ or 0.25	4	0.01 or 0.001	visual comparison of schemes
<code>convergence</code>	square meshes, $h = 0.5$ to 0.03125	2	scheme-dependent	spatial and temporal error rates
<code>gaussian_pulse</code>	square, $h = 0.0625$ or 0.125	2	0.01 or 0.001	pulse propagation and stability
<code>two_sided_driven_boundary</code>	square, $h = 0.0625$	2	0.01	time-dependent boundary forcing
<code>rectangle_driven_left_boundary</code>	rectangle, $h = 0.125$	2	0.01	non-square driven domain
<code>circle_standing_wave</code>	circle, $h = 0.125$	4	0.01	curved domain with exact-like data
<code>donut_angular_mode</code>	donut, $h = 0.125$	2	0.01	non-trivial topology

Table 4: Boundary functions g used in the configuration files.

Config name	Definition	Used for
<code>zero_dirichlet</code> or <code>zero</code>	$g = 0$ on $\partial\Omega$	standing wave, Gaussian pulse, convergence, circle and donut tests
<code>boundary_pulse_left</code>	On $x = 0$, $g = \sin(\pi y/5) \exp\left(-\frac{(y-2.5)^2}{2 \cdot 0.6^2}\right) \sin^2(\pi t/0.5)$ for $0 \leq t \leq 0.5$, zero otherwise	left-boundary pulse visualization
<code>rectangle_driven_left_boundary</code>	On $x = 0$, $g = 0.5 \sin(\pi y/5) \sin(2\pi t)$, zero otherwise	driven rectangular domain
<code>two_sided_driven_boundary</code>	On $x = 0$, $g = 0.5 \sin(\pi y/5) \sin(2\pi t)$ on $x = 5$, the same value with opposite sign, zero otherwise	two-sided driven square tests

The same factory also provides the initial data and forcing terms used by the experiments: the manufactured standing wave on $[0, 5]^2$, its velocity and forcing, the circular standing-wave data, a Gaussian pulse centered at $(2.5, 2.5)$, and the annular angular mode.

The code uses conjugate gradients with a Trilinos SSOR preconditioner for the SPD systems. Boundary values are imposed by `MatrixTools::apply_boundary_values` in solve systems, and by direct insertion when only a vector needs to be made consistent. For time-dependent Dirichlet data, boundary velocities and accelerations are approximated by finite differences of the prescribed boundary function. This is important in the driven examples, where the boundary itself injects energy into the computational domain.

5.2 Diagnostics and Output

The solver writes solution files for ParaView and can also write per-step diagnostics. The discrete energy used in the diagnostics is

$$E_h^n = \frac{1}{2}(\mathbf{v}^n)^T \mathbf{M} \mathbf{v}^n + \frac{c^2}{2}(\mathbf{u}^n)^T \mathbf{K} \mathbf{u}^n. \quad (15)$$

For the standing wave diagnostic, a probe at $(2.5, 2.5)$ is also logged and compared against the exact trace. This makes dissipation and dispersion easy to separate: amplitude loss appears directly in the energy and in the probe trace, whereas phase error appears as a time shift between the numerical and exact signals.

The repository also contains a `videos/` directory with ParaView visualizations for representative configurations. In particular, the videos show that Crank-Nicolson and Newmark avg accel give the cleanest and most stable wave propagation, while Leapfrog and Backward Euler show more visible numerical artifacts. Forward Euler is unstable when the time step is not compatible with the explicit stability restriction. The `pulse` subfolder contains runs at different time steps that make this instability visible.

6 Convergence Study

The convergence study uses the exact standing wave (5). Two separate sweeps are performed. The spatial sweep keeps the time step small and refines the mesh from `h0p5` to `h0p0625`. The temporal sweep uses a fixed fine mesh and varies Δt . The final time is $T = 1$ for most schemes, except for Newmark avg accel where the configured run uses $T = 5$.

Table 5: Convergence indicators. Spatial orders are reported for the last refinement step. Temporal orders are the representative asymptotic values before the curves hit the spatial floor or instability.

Scheme	Spatial L^2	Spatial H^1	Temporal L^2	Temporal H^1
Crank-Nicolson	2.02	1.01	about 2	about 2 on coarse Δt
Newmark avg accel	2.02	1.00	about 2	spatial floor for small Δt
Backward Euler	2.09	1.01	about 1.1	about 1.1
Newmark leapfrog	1.99	1.01	unstable at $\Delta t = 0.02$	spatial floor after stable steps
Forward Euler	unstable on finest point	unstable on finest point	unstable for tested large steps	unstable

The table is intentionally not cleaned up by removing all bad explicit runs. For a wave solver, failed stability cases are part of the numerical evidence: they show where the explicit discretization cannot be trusted. In contrast, the implicit conservative schemes display the expected rates without requiring manual filtering of divergent runs.

θ -method Crank-Nicolson ($\theta = 0.5$)

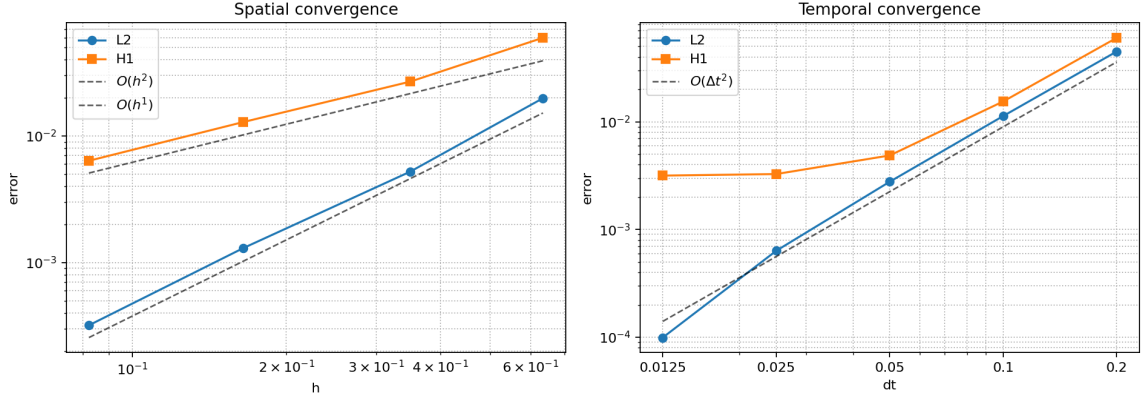


Figure 1: Convergence for Crank-Nicolson. The spatial behavior follows the expected $O(h^2)$ and $O(h)$ rates for P_1 elements. The L^2 temporal curve is close to second order, while the H^1 curve eventually reaches the spatial error floor.

Newmark average acceleration ($\beta = 0.25$, $\gamma = 0.5$)

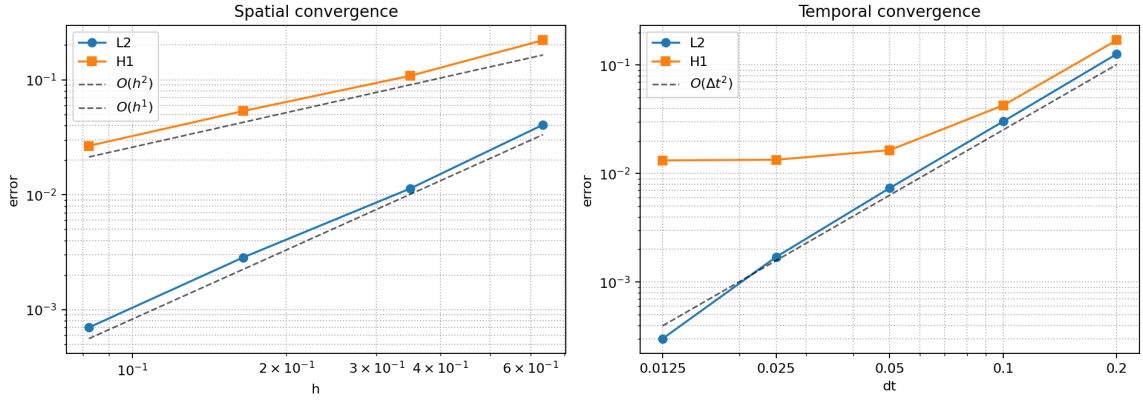


Figure 2: Convergence for Newmark avg accel. The method shows the same favorable behavior as Crank-Nicolson: optimal spatial convergence and approximately second-order temporal convergence before the spatial error dominates.

θ -method backward Euler ($\theta = 1$)

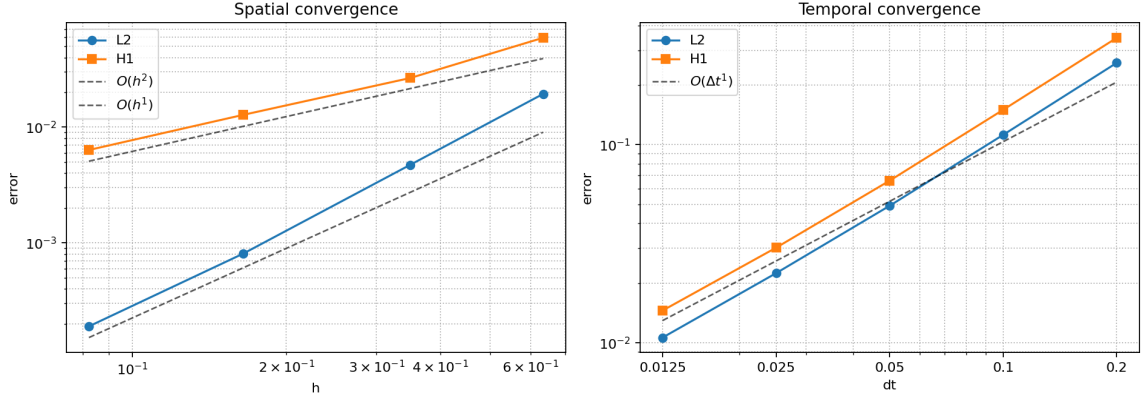


Figure 3: Convergence for Backward Euler. The spatial rates are close to the expected finite element rates when Δt is sufficiently small. The temporal plot exhibits the expected first-order behavior.

Newmark leapfrog ($\beta = 0$, $\gamma = 0.5$)

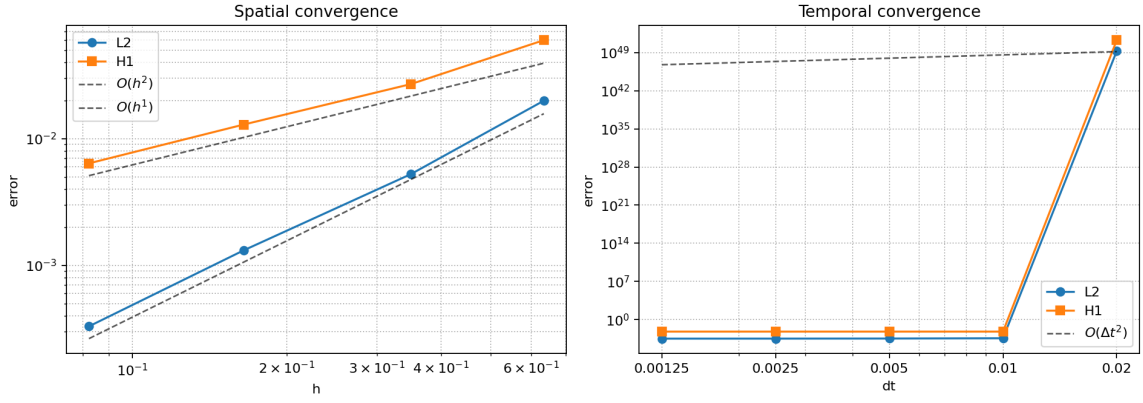


Figure 4: Convergence for Newmark leapfrog. The stable spatial sweep gives the expected P_1 finite element rates. In the temporal sweep, the largest time step is outside the stable range and the error grows to practically infinity. The remaining stable points are already dominated by the spatial error.

θ -method forward Euler ($\theta = 0$)

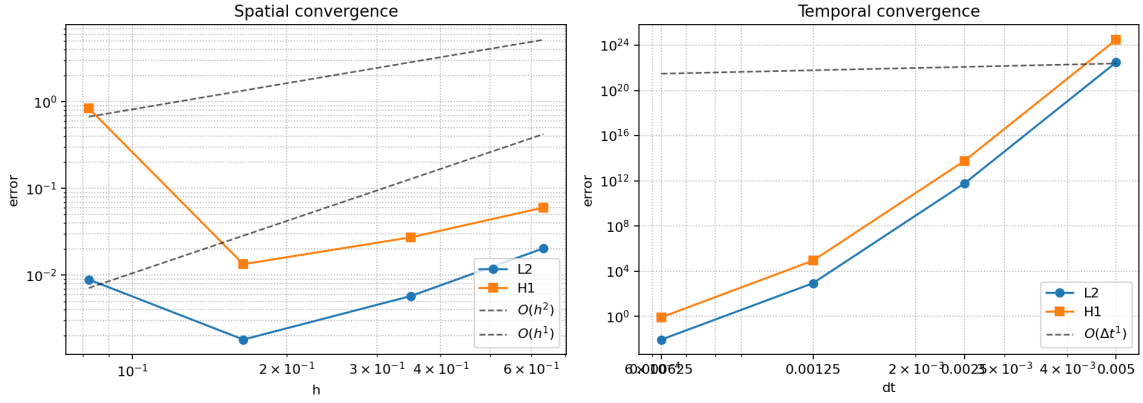


Figure 5: Convergence for Forward Euler. The explicit scheme is very sensitive to the explicit stability restriction in this implementation. The temporal sweep is dominated by unstable runs, and the finest spatial point also shows loss of stability rather than asymptotic convergence.

The cleanest convergence evidence is given by Crank-Nicolson, Backward Euler and Newmark avg accel. For Crank-Nicolson and avg accel, the observed L^2 spatial orders are close to two on the refined part of the curve, while the H^1 orders are close to one. This matches the expected finite element estimate

$$\|u - u_h\|_{L^2(\Omega)} = O(h^{r+1}), \quad \|u - u_h\|_{H^1(\Omega)} = O(h^r), \quad r = 1.$$

Backward Euler also exhibits the expected spatial rates but only first-order time convergence, as predicted by the method. The explicit schemes require more care: the plots are useful precisely because they show the stability restriction. Once the time step violates the admissible stability range, the error does not represent convergence anymore, it represents numerical divergence.

7 Dissipation and Dispersion

The dissipation and dispersion audit runs all five schemes on the same standing wave problem up to $T = 5$, with time steps

$$\Delta t \in \{0.15, 0.1, 0.05, 0.02, 0.01, 0.005, 0.001\}.$$

The mesh is `square_5x5_h0p25.msh`, the wave speed is $c = 4$, and the probe point is the center of the square. Since $\sin(\pi x/5) \sin(\pi y/5) = 1$ at $(2.5, 2.5)$, the exact probe is simply a cosine in time.

Discrete energy audit

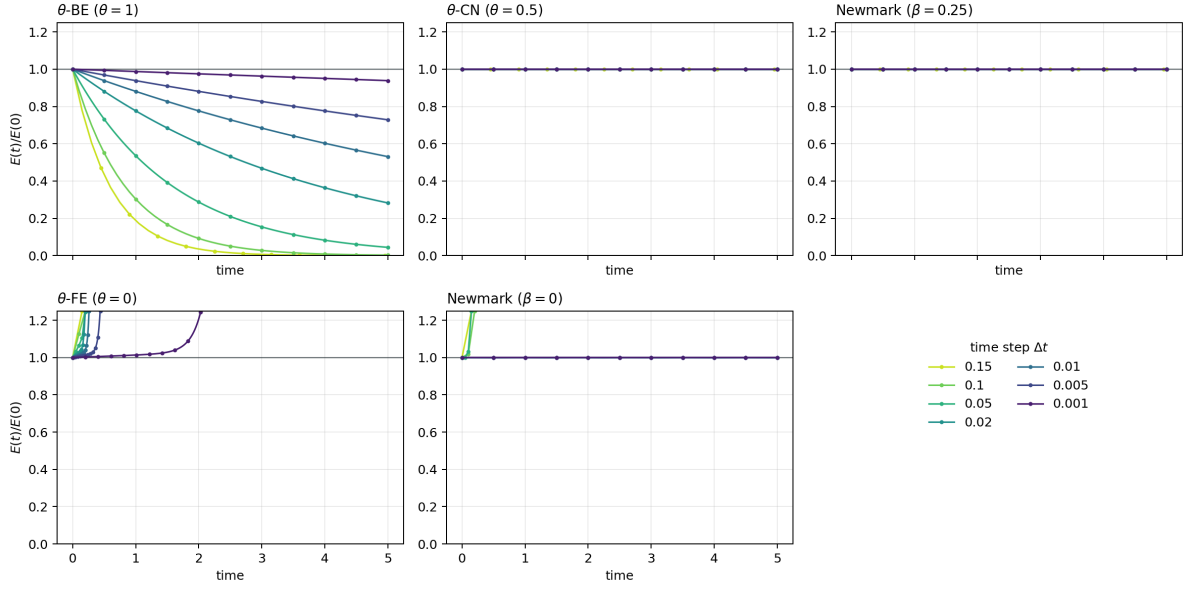


Figure 6: Normalized discrete energy $E_h(t)/E_h(0)$. Crank-Nicolson and Newmark avg accel preserve energy almost exactly. Backward Euler damps the solution, with stronger damping for larger Δt . Forward Euler and Newmark leapfrog diverge when the time step is outside the stable range.

Figure 6 is the clearest numerical confirmation of the theory. Crank-Nicolson and avg accel lie on the reference energy line for all tested time steps. Backward Euler is unconditionally stable but very dissipative: for $\Delta t = 0.15$ the final energy ratio is approximately $2.54 \cdot 10^{-4}$, and even for $\Delta t = 0.01$ only about 53% of the initial energy remains at $T = 5$. Forward Euler diverges for all time steps in this diagnostic sweep, while Newmark leapfrog is stable for $\Delta t \leq 0.02$ and diverges for the larger steps.

Point-probe trace against exact solution

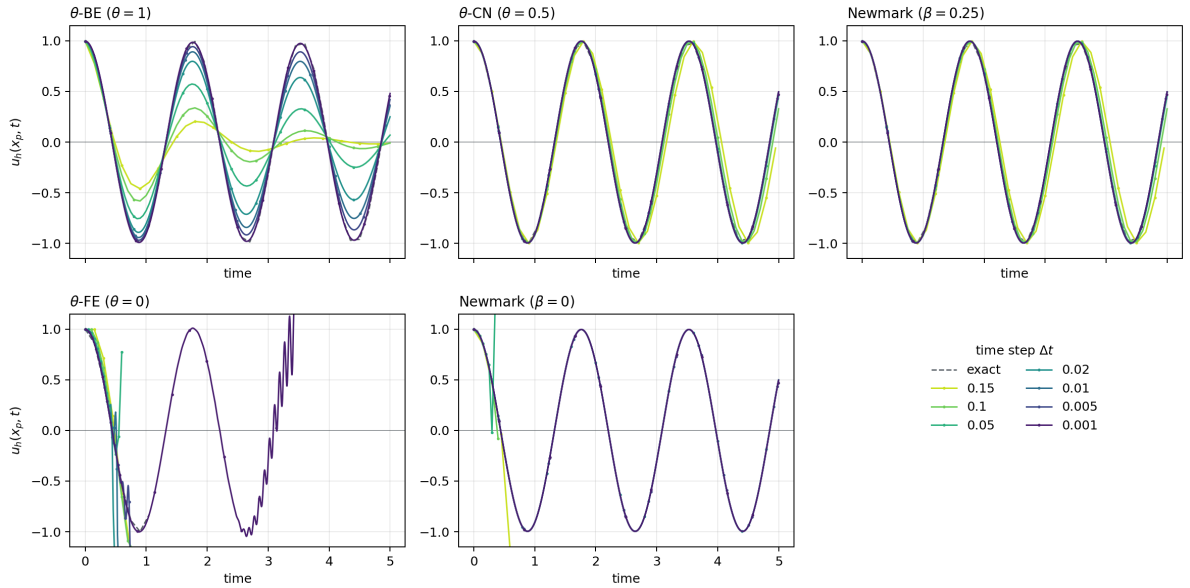


Figure 7: Value trace at $(2.5, 2.5)$ compared with the exact cos. Phase shifts indicate numerical dispersion, amplitude loss indicates dissipation.

The probe traces in Figure 7 separate amplitude and phase effects. Backward Euler visibly damps the oscillation and also shifts the phase. Crank-Nicolson and Newmark avg accel keep the amplitude almost perfectly, but large time steps introduce a visible phase lag. This is the typical behavior of non-dissipative second-order schemes for wave problems: the energy is preserved, yet the numerical wave speed is not exact. Leapfrog matches the exact trace well in the stable small-step cases, but its stability margin is narrow. Forward Euler is qualitatively unsuitable here because the energy grows instead of remaining bounded.

8 Additional Simulations

Beyond the quantitative convergence and diagnostic sweeps, the project includes several visualization-oriented configurations. The Gaussian pulse examples start from a localized initial displacement in the square and compare the time schemes visually. The two-sided driven boundary example excites the square from the left and right boundaries with opposite signs. The rectangle example drives the left boundary on the domain $[0, 8] \times [0, 5]$. The circular and annular examples show that the mesh-loading and boundary enforcement are not restricted to rectangular domains.

The videos in are meant to be opened separately. They support the same conclusions as the plots: Crank-Nicolson and Newmark avg accel produce the most reliable propagation, Backward Euler smooths and attenuates the wave, Leapfrog can be accurate but is more fragile, and Forward Euler can become unstable when the explicit time step is not acceptable for the mesh and wave speed.

Table 6: Video files used as qualitative ParaView checks.

Video group or file	What it illustrates
<code>standing-wave/</code>	Standing-wave propagation with stable schemes and a Forward Euler unstable case.
<code>pulse/</code>	Gaussian pulse comparison, including the stable <code>pulse-FE-0.01.mov</code> run and the unstable <code>pulse-FE-0.001.mov</code> run.
<code>two-sided/</code>	Boundary-driven square problem for FE, CN, BE, leapfrog and avg accel.
<code>boundary-pulse.mov</code>	Pulse-like boundary excitation and qualitative wave propagation.
<code>donut.mov</code>	Propagation on the annular mesh with an angular initial mode.
<code>rectangle-driven-left-boundary. mov</code>	Driven left boundary on the rectangular domain.

These files should be read as a visual sanity check rather than as a substitute for the quantitative plots. For example, the conservative schemes look better because their amplitude does not artificially decay. This agrees with Figure 6. Backward Euler remains stable but visibly loses amplitude, again matching its energy curve. Forward Euler and leapfrog are the ones where the videos are most useful: once the stability threshold is crossed, the growing oscillations are easier to recognize in the animation than in a single final-time error number.

9 Scalability

The scalability experiment available in the repository measures strong scaling for Newmark avg accel on a large square mesh with 452660 degrees of freedom. The run uses $\Delta t = 0.01$ and 10 time steps. The measured wall times are shown in Figure 8 and summarized in Table 7.

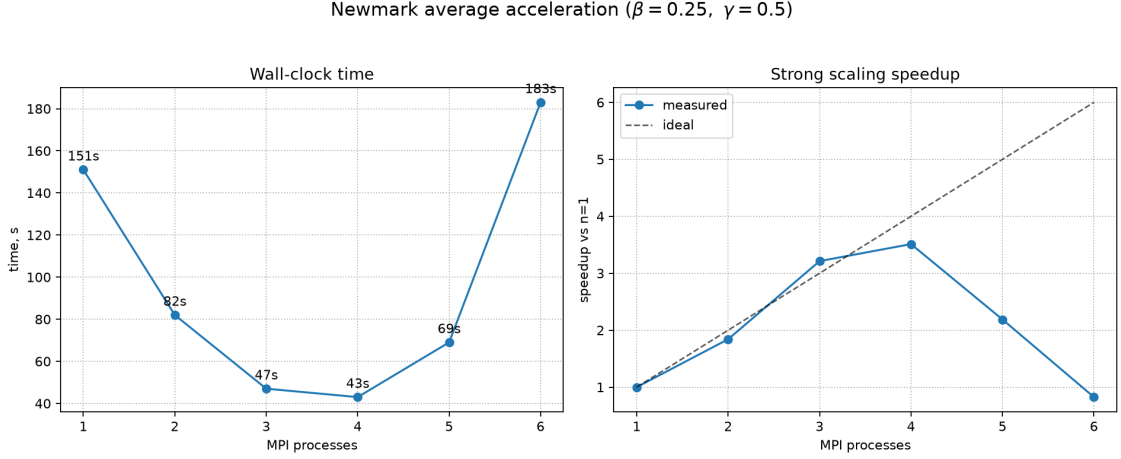


Figure 8: Strong scaling for Newmark avg accel. The solver scales well up to four MPI processes on this dataset, then performance degrades for five and six processes.

Table 7: Strong scaling data for Newmark avg accel.

MPI ranks	Wall time [s]	Speedup	Efficiency
1	151	1.00	1.00
2	82	1.84	0.92
3	47	3.21	1.07
4	43	3.51	0.88
5	69	2.19	0.44
6	183	0.83	0.14

The first four points show that the distributed vector operations and linear solves benefit from MPI parallelism. The superlinear speedup at three ranks is likely caused by cache and run-to-run effects rather than by the algorithm itself. The degradation after four ranks indicates that this particular run is no longer compute-dominated: communication, partitioning quality, synchronization and preconditioner overhead become too large compared with the amount of work assigned to each rank. Therefore the implementation is parallel, but the current benchmark should be interpreted as good strong scaling only up to four ranks for the tested hardware and problem size.

10 Limitations and Possible Improvements

The project already covers the requested numerical aspects, but several limitations are visible from the results. First, the explicit methods are not protected by an automatic explicit-stability estimator in the current configuration files. This is why the Forward Euler and leapfrog sweeps contain divergent points. A practical improvement would be to compute a conservative estimate of $h_{\min}/c$ after mesh loading and reject explicit runs whose time step is too large. The report intentionally keeps the unstable data because it documents the actual behavior of the solver, but an automated guard would make production runs safer.

Second, the explicit methods still solve systems involving the mass matrix. This is acceptable for a unified implementation and for the project scale, but it is not the most efficient version of an explicit wave solver. A mass-lumped formulation would replace these solves by diagonal scaling and would make Forward Euler or leapfrog much cheaper per time step. The trade-off is that mass lumping changes the spatial discretization and may affect dispersion, so it should be evaluated with the same energy and probe diagnostics used in this work.

Third, the scalability study is limited to one method and one problem size. It is still useful because it confirms that the distributed data structures work, but it does not fully characterize

the solver. A more complete performance study would repeat the strong-scaling experiment for all schemes, include weak scaling, and separate setup, assembly, solve and output times. Such a split would make it easier to distinguish communication overhead from linear solver cost and from file I/O.

Finally, all reported convergence runs use linear finite elements. The code allows `fe_degree > 1`, so a natural extension would be to repeat the study with quadratic elements. This would test the expected $O(h^3)$ L^2 behavior and would also make dispersion effects smaller for the same mesh size.

11 Conclusions

The implemented solver follows the standard finite element semi-discretization of the wave equation and provides two families of time integration schemes. The numerical results are consistent with the main theoretical expectations. Linear finite elements give approximately second-order convergence in L^2 and first-order convergence in H^1 when the time discretization is stable and the temporal error is not dominant. Backward Euler is first-order in time and strongly dissipative. Crank-Nicolson and Newmark avg accel are the best qualitative choices among the tested methods because they preserve the discrete energy and remain stable over long simulations, their main remaining error is dispersion for large time steps. Leapfrog can be accurate and energy-preserving in the stable regime but is limited by the restriction $\Delta t \leq Ch$. Forward Euler is the least robust method and shows instability in the diagnostic runs.

From an implementation perspective, the project supports reproducible configuration-driven runs, external Gmsh meshes, distributed deal.II triangulations, strong Dirichlet boundary enforcement, ParaView output, convergence sweeps and energy diagnostics. The videos and plots agree: Newmark avg accel and Crank-Nicolson give the most stable visual results, while the dissipative or explicit schemes show the expected artifacts. Overall, Newmark avg accel offers the best balance in this codebase: it is stable, energy-preserving, visually clean and supported by the strongest convergence and diagnostic evidence.

References

- [1] Alfio Quarteroni. *Numerical Models for Differential Problems*. Springer, 2009.
- [2] Sandro Salsa. *Partial Differential Equations in Action: From Modelling to Theory*. Springer, 2008.